



TECHNICAL ARCHITECTURE REPORT

The platform's application framework, hosting, data layer, runtime, engineering practice and real-time delivery, stated with the versions actually in service.

PREPARED BY	Ocean Entertainment Limited
PREPARED FOR	Gaming Board of Tanzania
PLATFORM	50pick · 50pick.tz
DATE	24 August 2026
VERSION	1.0
CLASSIFICATION	Confidential — regulatory submission

CONTENTS

Twelve sections and two appendices. Sections 02 to 08 answer the Board's technical schedule directly; 09 to 12 cover the controls that surround it.

01	Scope & summary of the stack
02	Application framework
03	Cloud hosting provider
04	Database technology
05	Server & runtime
06	Programming languages
07	Development & quality assurance
08	Real-time messaging
09	Security architecture
10	Data residency, protection & retention
11	Resilience, backup & continuity
12	Observability & monitoring
A	Component version register
B	Glossary of technical terms

01

SCOPE & SUMMARY OF THE STACK

What this report covers, and the Board's technical schedule answered in a single table before each item is expanded.

This report describes the technology on which 50pick runs. It is written from the system in service rather than from a design document: every version, count and configuration value in it was read from the running platform or from the source repository on the date on the cover.

It accompanies the *50pick Platform Profile* (ref 50P-PP-2026-01), which covers registration, identity verification, funds movement, game mechanics and operational controls. Where that document explains what the platform does, this one explains what it is built on.

TABLE 1 — *The Board's technical schedule, answered*

ITEM	ANSWER IN SERVICE
Framework	Next.js 16.2.4 (App Router) with React 19.2.5 and Tailwind CSS 3.4.19. §02
Cloud hosting provider	Railway for the application, database and cache; Cloudflare for domain routing and encrypted object storage. §03
Database technology	PostgreSQL 18.6, managed, accessed exclusively through the Prisma 6.19.3 ORM. §04
Server	Node.js 24 LTS running the Next.js production server, with Redis for coordination between instances. §05
Programming languages	TypeScript 5.9.3 for the application, SQL for schema and migrations, HTML and CSS for presentation. §06
Development	Continuous delivery on Git and GitHub, with 243 automated test suites and 79 mutation harnesses gating every change. §07
Real-time messaging	Server-Sent Events over HTTPS, fanned out across instances by Redis publish/subscribe. §08

The shape of the system

50pick is a single application. One codebase renders the player-facing site, serves the administration console and exposes the platform's programmatic endpoints, which means there is one place where a rule about money is written and one place it is enforced. The application is stateless: every instance is interchangeable, and all state lives in the database, the cache or the object store.

TABLE 2 — *The platform by size, as at the date of this report*

MEASURE	COUNT
Application source files	774
Player and console page routes	94
Programmatic endpoints	49
Database tables	43
Enumerated types	36
Versioned schema migrations	62
Interface strings, per language	1,918 × 3
Automated test suites	243
Mutation harnesses	79

02

APPLICATION FRAMEWORK

Next.js 16 on the App Router, with React 19 server components. Pages are assembled on the server, so a player's device is never trusted with a decision.

TABLE 3 — Framework components

COMPONENT	VERSION	ROLE
Next.js	16.2.4	Application framework, routing, server rendering, server actions
React	19.2.5	Component model, server components
Tailwind CSS	3.4.19	Styling, bound to a fixed set of design tokens
Zod	3.25.76	Schema validation at every trust boundary

Why server rendering matters for a gambling platform

Every page that shows money, and every action that moves it, is produced on the server. A player's browser receives finished output; it is never the authority on a balance, a stake limit, a market's status or a payout. An altered request from a modified browser reaches the same server-side rule as an unaltered one and is refused by it.

The same principle governs the administration console. A member of staff whose role does not permit an action is prevented from taking it by a check that runs on the server for that specific action, not by hiding the button. A control that is not shown and a control that is not permitted are two different things, and the platform enforces the second.

One interface, three languages

The platform is delivered in English, Swahili and Chinese. Interface text is held in a single dictionary of 1,918 keys per language rather than written into individual screens, and an automated check refuses any change that leaves the three languages out of step. A player's language governs every screen, notification and email they receive, not only the page they chose it on.

Design consistency is enforced, not encouraged

Typography, spacing, colour and component usage are held to a closed set of tokens. Automated checks reject a change that introduces a value outside that set, so the interface cannot drift screen by screen into inconsistency as it grows. The same checks confirm that each page declares its width from the shared scale and that no page duplicates a structural landmark, which is what keeps the platform navigable on a screen reader.

NOTE — §02

Mobile first. *The platform is built for the handset first and verified at three widths in all three languages before any interface change is released. Tanzanian players reach 50pick predominantly by phone, and the narrow layout is the one that is designed rather than the one that is inherited.*

03

CLOUD HOSTING PROVIDER

Railway hosts the application, its database and its cache. Cloudflare provides domain routing and the encrypted object storage that holds identity documents and backups.

TABLE 4 — Infrastructure providers and what each carries

PROVIDER	ROLE	WHAT IT HOLDS
Railway	Application, database, cache	The Next.js application instances, the managed PostgreSQL database, and the Redis instance used for coordination
Cloudflare	Domain routing, object storage	DNS for 50pick.tz; encrypted identity documents and encrypted database backups, held in separate stores under separate credentials
Selcom	Payment aggregator	Mobile-money collections and disbursements across M-Pesa, Airtel Money, HaloPesa and Mixx by Yas
Postmark	Transactional email	Account, security and money-movement messages
Sentry	Error monitoring	Application error and performance telemetry, in a European region

Transport security

The platform is served over HTTPS only. Strict Transport Security is declared with a two-year maximum age, applied to every subdomain, and submitted to the browser preload list — so a browser that has never visited 50pick.tz will still refuse to make an unencrypted first request. Connections between the application and its database are encrypted with TLS 1.3.

HSTS `max-age=63072000; includeSubDomains; preload`

DATABASE TRANSPORT `TLS 1.3`

Deployment

A release is produced from a commit in the source repository, built into an immutable container image and started only after its database migrations have applied successfully. Nothing is edited in place on a running server, and no change reaches production without passing through the automated checks described in Section 07.

The platform runs as more than one instance. Instances are interchangeable and hold no state of their own, so one can be replaced without a player noticing. Work that must not happen twice is coordinated as described in Section 11.

Separation of credentials

Credentials exist only in the deployment environment and never in source code. The document store and the backup store use different credentials, so a key that reached one would not open the other. Application errors are reported to a monitoring service that is configured to receive diagnostic context, not player records.

04

DATABASE TECHNOLOGY

PostgreSQL 18.6, managed and encrypted in transit, reached only through a typed data-access layer. Every movement of money is written twice and reconciled against itself.

TABLE 5 — Database configuration in service

PROPERTY	VALUE
Engine	PostgreSQL 18.6
Character encoding	UTF-8
Collation	en_US.utf8
Connection transport	TLS 1.3
Access layer	Prisma ORM 6.19.3
Tables	43
Applied migrations	62
Connection pool ceiling	40

Schema change control

The schema is versioned. Each change is a numbered, immutable migration held in the source repository, reviewed before merge and applied automatically when the release that needs it starts. Migrations move forward only and none is edited after it has been applied, so the sequence of 62 is a complete, dated history of how the data model reached its present state.

How money is written

A player's balance is a recorded liability, not a running total that is simply adjusted. Every stake, payout, refund, deposit and withdrawal is written twice: once against the wallet balance and once as a balanced set of double-entry ledger postings. The two are reconciled against each other continuously, so a balance that disagrees with its own history is detected rather than carried forward.

Each of those writes happens inside a single database transaction. A transaction either completes in full or leaves no trace, which is what makes a partial settlement impossible rather than merely unlikely.

Preventing two things happening at once

Where two operations could touch the same money at once — two bets on one market, a settlement running while a player cashes out — the platform takes a database-level advisory lock keyed to that wallet or market. The lock is held by the database, so it is honoured across every application instance rather than only within one. Requests arriving while it is held are queued and served in order: load presents as a short wait, never as a lost or duplicated transaction.

NOTE — §04

Verified under load. *The concurrency design has been measured rather than assumed: two hundred simultaneous bets placed on a single market all succeeded, the pool balanced exactly against the sum of stakes, and no money was created or lost.*

05

SERVER & RUNTIME

Node.js 24 LTS running the Next.js production server, with Redis coordinating the instances behind it.

TABLE 6 — Runtime components

COMPONENT	VERSION	ROLE
Node.js	24 LTS	JavaScript runtime; the same major version is pinned in continuous integration
Next.js server	16.2.4	Serves every page, endpoint and server action
Redis (ioredis)	5.11.1	Cross-instance messaging, rate-limit counters, coordination
PostgreSQL driver	pg 8.22.0	Direct database access for operational tooling

Long-lived state does not live in the server

Application instances hold nothing that matters. Sessions are carried in signed cookies and validated against the database; counters and locks live in Redis and PostgreSQL. An instance can therefore be replaced, restarted or added at any moment without interrupting a player mid-action, and without any single machine becoming irreplaceable.

Scheduled work runs exactly once

Some work is time-driven rather than player-driven: settling a market when its objection window closes, reconciling a payment the network has confirmed, closing an expired round. With several instances running, such a task must not be performed more than once.

The platform elects a single instance to hold a time-limited lease. Only the leaseholder performs scheduled money work, and the lease expires by itself, so if that instance stops, another takes over without intervention and without any risk of both acting at once.

Capacity

Database connections are pooled with a ceiling of 40 per instance, and requests that would exceed the ceiling wait in an orderly queue rather than failing. The design decision throughout is that pressure should present as latency, never as an error shown to a player and never as a transaction in an uncertain state.

Health reporting

The platform exposes an internal health endpoint that reports the live state of the database, the email provider, the payment subsystem, the cache and which instance currently holds the scheduling lease. A failed dependency is therefore visible immediately rather than being discovered through a player complaint.

06

PROGRAMMING LANGUAGES

TypeScript throughout the application, SQL for the data model, and HTML and CSS for presentation. One language on both sides of the network.

TABLE 7 — Languages in use

LANGUAGE	VERSION	WHERE IT IS USED
TypeScript	5.9.3	The entire application — server logic, money arithmetic, interface, administration console and automated tests
SQL	PostgreSQL 18	Schema definition, the 62 versioned migrations, and reporting queries
HTML & CSS	—	Presentation, generated from components rather than authored per page

Why a typed language was chosen

TypeScript checks the shape of every value before the code runs. A payout calculated in shillings cannot be passed where a percentage is expected; a market that may not have been settled cannot be treated as though it certainly has. These are not warnings — the build fails and the release does not proceed.

The practical consequence for a gambling platform is that a whole class of arithmetic and null-value defects is eliminated before any code is executed, rather than discovered by a player whose balance is wrong.

One definition of the money rules

The arithmetic that decides a payout is written once, in a single module, and imported by both the server that settles a market and the screen that shows a player what they stand to win. It is not possible for the figure quoted before a bet and the figure paid after it to be produced by two different pieces of code, because there is only one.

Validation at the boundary

Every value that enters the platform from outside — a form submission, a payment notification, an administrative action — is parsed against an explicit schema before it is used. Anything that does not match the declared shape is rejected at the boundary with a stated reason, rather than being allowed part-way in and failing later somewhere less obvious.

07

DEVELOPMENT & QUALITY ASSURANCE

Continuous delivery, with every change gated by an automated suite that includes tests written specifically to attack the platform's own safety checks.

How a change reaches players

- 1 **Written and reviewed.** Every change is made against the source repository on GitHub and reviewed before it is merged.
- 2 **Checked automatically.** Continuous integration runs on every commit: type checking, the test suites, interface-consistency checks, translation parity and file encoding.
- 3 **Verified in a browser.** Browser tests drive the real application the way a player would, at three screen widths and in all three languages.
- 4 **Released.** A passing commit is built into an immutable image, its database migrations applied, and the instance started only if they succeeded.
- 5 **Confirmed live.** The release is exercised against the running platform before the change is considered delivered.

TABLE 8 — *The automated gate*

CHECK	COUNT	WHAT IT PROTECTS
Automated test suites	243	Money conservation, settlement arithmetic, fee models, identity rules, access control, responsible-gambling limits
Mutation harnesses	79	Prove each safety check is capable of failing — see below
End-to-end suites	3	Complete player journeys against a real database
Live verification drivers	79	Exercise the running platform in a real browser
Operational tooling	40	Audited, read-only or single-purpose administrative procedures

Mutation testing

A test that passes proves only that the system behaves correctly today. It does not prove the test would notice if the system stopped behaving correctly. A check that can no longer fail is indistinguishable from a check that is working, and it is the more dangerous of the two because it is trusted.

50pick therefore maintains 79 mutation harnesses. Each one takes a copy of the platform, deliberately introduces a specific defect into it — removes a limit, inverts a comparison, disables a lock — and then requires that the corresponding safety check detects it, by name. A defect that goes undetected is treated as a finding against the check, not against the experiment.

This is applied to the controls that matter most: that a pool always balances to zero, that a fee is never taken from a refunded stake, that a bet after the close is refused, that a withdrawal above the review threshold cannot be released by one officer, and that a limit a player has set is enforced by the server.

NOTE — §07

Every figure in a released document is re-derived. *Counts, versions and financial figures quoted in operator documentation are read from the running system when the document is produced, rather than carried forward from a previous issue. A number restated from memory is a number that eventually disagrees with the system it describes.*

08

REAL-TIME MESSAGING

Server-Sent Events over HTTPS deliver live updates to the player's screen; Redis publish/subscribe carries them between application instances.

TABLE 9 — *The real-time channel*

PROPERTY	VALUE
Transport to the browser	Server-Sent Events over HTTPS
Content type	text/event-stream
Authentication	Session validated before the stream opens; refused without one
Keep-alive	15-second heartbeat
Between instances	Redis publish/subscribe on a dedicated channel
Reconnection	Automatic, by the browser

What travels over it

Live market and round prices as stakes join either side; the result of a round the moment it settles; a player's own wallet balance when it changes; and notifications addressed to that player. Messages are typed, and the channel accepts only the fixed set of message types the platform defines.

Messages are scoped. A balance update is delivered to the account it belongs to and to no other; market and round updates are public information and are broadcast. A connected browser cannot subscribe to another player's private stream.

Why Server-Sent Events rather than WebSockets

The platform's live data flows in one direction: from the server to the player. Every action that flows the other way — placing a stake, cashing out, requesting a withdrawal — is a separate authenticated request that must be validated, rate-limited and audited in its own right, and would be neither safer nor faster over a persistent socket.

For one-way delivery, Server-Sent Events are the more robust choice on the networks Tanzanian players actually use. The channel is ordinary HTTPS, so it passes through corporate proxies and mobile network intermediaries that interfere with socket upgrades; the browser reconnects on its own after a signal drop without the platform maintaining reconnection logic; and it inherits the same transport security, the same authentication and the same rate limiting as every other request rather than requiring a parallel set.

Delivery across instances

A player's browser is connected to one application instance, but the event that concerns them may be produced by another — a settlement, for example, is performed by whichever instance holds the scheduling lease. Each instance therefore publishes its events to a Redis channel, and every instance subscribes to it and forwards what is relevant to the players connected to it.

The design accounts for the fact that a publisher also receives its own message: an event is emitted locally once and relayed once, and a message arriving back from the channel is not republished. A settled round therefore reaches every connected player exactly once, whichever instance settled it.

NOTE — §08

The channel is a convenience, never an authority. *A live update tells a player's screen that something has changed; it never decides what changed. Balances, results and payouts are read from the database and settled by the server, so a player who loses connectivity sees the same outcome on reload as one who stayed connected throughout.*

09

SECURITY ARCHITECTURE

Independent layers, each addressing a different way an account or a transaction could be misused.

TABLE 10 — Technical security controls

CONTROL	IMPLEMENTATION
Password storage	script with a unique per-account salt. The password itself is never stored and the stored value cannot be reversed.
Session integrity	Cookies signed with HMAC-SHA-256, marked HTTP-only and rotated on each sign-in. An altered cookie fails verification and is refused.
One-time codes	Stored hashed with a server-side pepper; never held in readable form.
Rate limiting	Token-bucket throttling applied per account and per action across sign-in, one-time codes and other sensitive operations.
Timing safety	Sign-in performs equivalent work whether or not an account exists, so response time does not reveal which phone numbers are registered.
Two-factor authentication	One-time-code verification available to players and to staff using the administration console.
Role-based access	Staff permissions are granted per role and per domain and are checked on the server for each action.
Audit chain	Every privileged action is appended to a record in which each entry is cryptographically linked to the one before it, so a later edit or deletion is detectable.

Browser-level protections

The platform sets security headers on every response. A content security policy constrains where the page may load resources from and where it may submit to; framing is refused outright, which prevents the site being embedded in a deceptive page; content-type sniffing is disabled; referrer information is limited; and device capabilities the platform does not use — microphone, geolocation, payment handlers, USB — are switched off at the browser level.

FRAMING	frame-ancestors 'none' · X-Frame-Options: DENY
FORM SUBMISSION	form-action 'self' · base-uri 'self'
CONTENT SNIFFING	X-Content-Type-Options: nosniff
REFERRER	strict-origin-when-cross-origin
TRANSPORT	upgrade-insecure-requests

Separation of duties

The platform's most consequential actions are structured so that one person acting alone is not sufficient. A withdrawal at or above the review threshold requires two different compliance officers to approve it before any payment instruction is created, and a stricter two-officer market-resolution mode is available as a configurable option. Both are enforced by the server, not by procedure.

Secrets

Credentials and signing keys exist only in the deployment environment, never in source code or in a build artefact, and are separated by purpose so that a single exposed key does not open an unrelated system.

10

DATA RESIDENCY, PROTECTION & RETENTION

Where data is held, how it is protected, and how long each kind of record is kept.

Where data is held

The application and its database run in a United States region. Identity documents and database backups are held in a Western European region, in two separate stores under separate credentials. Both are encrypted at rest.

Identity documents

Document images are stored as encrypted objects and are never served directly. A compliance officer viewing one receives a signed link valid for a short, fixed period, and every view or download is written to the permanent record with the officer's identity and a timestamp.

TABLE 11 — *Retention periods*

RECORD	RETAINED
Transactions, ledger entries, bet history	7 years
Identity documents and decisions	7 years from account closure
The permanent action log	7 years, not editable or deletable early
In-app notifications	180 days, then purged automatically
Marketing-consent records	2 years from last activity

A player's own data

A player can export a complete copy of their own data from their profile at any time. Requests to correct or erase personal data are submitted the same way and reach a compliance officer within the statutory period.

Erasure is handled so that it does not defeat a control that exists to protect other players. Because the identity-document number is what prevents one person opening several accounts, a closed account's number is irreversibly transformed into a value that can still be compared against a future submission but can no longer be read. The document images themselves are deleted outright.

NOTE — §10

Retention is enforced by mechanism, not by policy alone. *Records subject to the seven-year requirement are held in structures that no automated process is permitted to delete, and the permanent action log is append-only by construction rather than by convention.*

11

RESILIENCE, BACKUP & CONTINUITY

The platform is designed to lose a component without losing a transaction, and every backup is proven by restoring it.

Backups

The database is backed up nightly on an automated schedule, encrypted, and shipped to storage separate from the live system. Each backup is then restored into a disposable database and checked against the original before it is trusted. A backup that has never been restored is an assumption; one that has been restored and verified is a fact, and the platform treats the distinction as material.

The restore procedure has been rehearsed end to end against a full production-scale copy, so recovery is a practised operation rather than a first attempt made under pressure.

TABLE 12 — Continuity measures

MEASURE	HOW IT WORKS
Multiple instances	The application runs as more than one interchangeable instance; any one can be replaced without interrupting play.
Single execution of scheduled work	Time-driven money work is claimed by one instance under a self-expiring lease, so it runs exactly once and resumes automatically if that instance stops.
Forward-only migrations	An instance starts only after its schema migrations have applied successfully, so a release can never serve traffic against a schema it does not expect.
Nightly verified backup	Encrypted, stored separately, and restore-checked before being trusted.
Payment-route health	A route that cannot be reached is closed to players in their own language rather than accepting a request the platform cannot fulfil.
Uncertain payouts held open	A payout the network can neither confirm nor deny is never assumed failed; the position is held rather than risking a double payment.

Degrading safely

Where a dependency is unavailable, the platform's response is to refuse clearly rather than to proceed uncertainly. A withdrawal cannot be requested against a payment route that is down. A market cannot be published against a source that has not been approved. A settlement that cannot be completed in full is not completed at all.

The principle behind each of these is the same: in a system holding players' money, an honest refusal is always preferable to an action whose outcome is unknown.

12

OBSERVABILITY & MONITORING

Faults are surfaced by instrumentation rather than by players reporting them.

TABLE 13 — *Monitoring surfaces*

SURFACE	WHAT IT REPORTS
Error monitoring	Application errors and performance telemetry, captured by Sentry and tagged by environment. Hosted in a European region.
Internal health endpoint	Live state of the database, cache, email provider and payment subsystem, plus which instance holds the scheduling lease.
Audit chain	A durable, tamper-evident record of every privileged action, which doubles as the forensic trail for any later investigation.
Operator alerts	Administrators are notified automatically when a monitored threshold is approached and again when it is reached.
Payment reconciliation	Deposits and withdrawals are re-checked against the payment network on a repeating cycle, so a payment left in an uncertain state is resolved rather than stranded.

Diagnosing a failure after the fact

When a deposit fails, the payment network's own reason code and message are recorded in the audit payload rather than discarded. This matters because a failure that leaves no explanation cannot be investigated later; the record survives log rotation and can be produced for a specific player and a specific transaction on request.

What monitoring does not receive

Error telemetry carries the technical context needed to diagnose a fault. It is not a channel for player records, identity documents or financial history, all of which remain within the platform's own database and object storage under the controls described in Section 10.

NOTE — §12

Reconciliation is continuous, not periodic. *The double-entry ledger and the wallet balances are checked against each other on a running basis rather than in an end-of-day batch, so a discrepancy is detected close to the moment it would have arisen rather than after a day of activity has been layered on top of it.*

A

COMPONENT VERSION REGISTER

The exact version of every principal component in service on the date of this report, read from the running platform and from the deployed dependency tree.

TABLE 14 — Principal components

COMPONENT	VERSION	LAYER
PostgreSQL	18.6	Database engine
Node.js	24 LTS	Server runtime
Next.js	16.2.4	Application framework
React	19.2.5	Component model
TypeScript	5.9.3	Language
Prisma ORM	6.19.3	Data access
Tailwind CSS	3.4.19	Styling
ioredis	5.11.1	Cache and messaging client
node-postgres	8.22.0	Database driver
Zod	3.25.76	Input validation
AWS S3 client	3.1089.0	Object storage access
Postmark client	4.0.7	Transactional email
Sentry	10.69.0	Error monitoring
Playwright	1.59.1	Browser-based verification

NOTE — APPENDIX A

This register is version-controlled with the platform. *Dependency versions are pinned in the source repository and resolved identically on every build, so the components listed above are the components that run. The register is reissued whenever a principal component changes.*

B

GLOSSARY OF TECHNICAL TERMS

Definitions of the technical terms used in this report.

TABLE 15 — Glossary

Framework	The foundation an application is built on, providing routing, rendering and structure so that each feature is not built from first principles.
Server rendering	Producing a page's finished content on the server before sending it, rather than assembling it on the player's device.
ORM	Object-relational mapper. A typed layer between application code and the database that removes hand-written query strings and the errors they carry.
Migration	A numbered, immutable change to the database structure, applied in sequence and recorded permanently.
Transaction	A set of database changes that either all succeed or all fail, leaving no partial state.
Advisory lock	A lock held by the database on a named resource, honoured by every application instance, so two operations cannot act on the same money at once.
Double-entry ledger	An accounting record in which every movement is written as balanced entries, so the books can be proven to reconcile.
Server-Sent Events	A standard HTTP mechanism for a server to push a continuous stream of updates to a browser, with automatic reconnection.
Publish/subscribe	A messaging pattern in which one component publishes an event and every interested component receives it, used here to carry events between instances.
scrypt	A password-hashing function designed to be expensive to attack in bulk. Recommended by OWASP for password storage.
HMAC	A keyed cryptographic signature. Used here to sign session cookies and to link each audit entry to the one before it.
Continuous integration	Automated checks that run on every change before it can be merged or released.
Mutation testing	Deliberately introducing a defect to confirm that the test intended to catch it actually does.
Lease	A time-limited claim held by one instance, expiring by itself, used to guarantee scheduled work runs exactly once.

DOCUMENT CONTROL

50pick Technical Architecture Report, version 1.0, reference 50P-TR-2026-01, issued 24 August 2026.
Prepared by Ocean Entertainment Limited for the Gaming Board of Tanzania. This report is reissued whenever a principal component or an architectural control it describes materially changes; the version and reference on the cover identify this issue.